WekaProject: Multilayer Perceptron Classification and Learning-Rate Experiments

Course : Artificial Intelligence System (Sistem Kecerdasan Buatan)

Student : Yoga Arfiansyah Firdaus

NIK : 202022610037

Program : Master of Information Systems, Telkom University

Abstract

This project implements a Multilayer Perceptron (MLP) classifier in Java using the WEKA machine-learning library and provides a Swing graphical user interface (GUI) for configuring and running experiments. The classification task uses a structured vehicle dataset with 99 records, three predictor attributes (wheels, chassis, and pax), and a nominal target attribute (vtype) with four classes: Bus, Car, Truck, and Bike. The baseline configuration uses a learning rate of 0.3, momentum of 0.2, 500 epochs, automatic hidden-layer sizing, classifier seed 0, and stratified 10-fold cross-validation with cross-validation seed 1. The baseline run reproduced the supplied WEKA Explorer results: 99 of 99 instances correctly classified (100% accuracy), Kappa 1.000, mean absolute error (MAE) 0.01838, and root mean squared error (RMSE) 0.02708. A learning-rate experiment compared values of 0.1, 0.3, and 0.5. Accuracy increased from 98.99% at 0.1 to 100% at 0.3 and remained at 100% at 0.5. MAE and RMSE decreased across the tested values. These findings apply to this dataset and experimental setup; they do not establish general performance on larger or unseen real-world vehicle data.

Keywords: Multilayer Perceptron, WEKA, Java, classification, cross-validation, learning rate, Swing GUI.

1. Introduction

Machine learning enables a computer to learn patterns from labelled examples and use those patterns to predict a target class for new observations. This project explores supervised classification using a Multilayer Perceptron (MLP), a feed-forward artificial neural network.

The project has two connected objectives. First, it integrates WEKA's MLP classifier into a Java application and provides a Swing GUI for selecting a dataset, setting model parameters, and displaying evaluation results. Second, it investigates how different learning rates affect the model's classification performance and prediction errors while other major settings are held constant.

The vehicle-classification dataset is used as a controlled educational example. The work demonstrates a machine-learning workflow and parameter comparison rather than delivering a production-ready vehicle-recognition system.

2. Objectives

1. Implement an MLP classifier using the WEKA Java API.
2. Load and validate an ARFF dataset and set the final attribute as the class attribute.
3. Evaluate the baseline model using stratified 10-fold cross-validation.
4. compare learning rates of 0.1, 0.3, and 0.5 while keeping the other model settings constant.
5. Present the experiment controls and results through a Java Swing GUI.
6. Interpret classification metrics and document limitations of the experiment.

3. Theoretical Background

3.1 Multilayer Perceptron

An MLP is a neural network made up of an input layer, one or more hidden layers, and an output layer. Connections between neurons have weights. During training, the network adjusts those weights to reduce prediction error. In this project, the input attributes are wheels, chassis, and pax; the output classes are Bus, Car, Truck, and Bike.

WEKA handles the internal encoding and model-training operations. The application configures the classifier and evaluates it through WEKA's API.

3.2 Learning rate and momentum

The learning rate controls the step size used when updating network weights during training. A rate that is too small may make learning slower, while a larger rate may improve or destabilize training depending on the data and other settings. There is no universally optimal learning rate.

Momentum incorporates information from previous weight updates to influence the next update. The experiments hold momentum at 0.2 so that the main parameter under investigation is the learning rate.

3.3 Evaluation metrics

- Accuracy: the proportion of observations classified correctly.
- Kappa: agreement between predicted and actual classes adjusted for chance agreement.
- Mean Absolute Error (MAE): average absolute difference between predicted class probabilities and the actual class indicator values.
- Root Mean Squared Error (RMSE): square root of the mean squared probability error; it penalizes larger errors more strongly than MAE.
- Confusion matrix: a table of actual versus predicted classes. Correct classifications appear on the diagonal.

MAE and RMSE are probability-based measures in this classification task; they are not simply the percentage of incorrectly classified records.

4. Dataset and Experimental Setup

4.1 Dataset

The reference experiment uses vehicledata_99.arff, containing 99 instances and four attributes:

Attribute	Role
`wheels`	Predictor
`chassis`	Predictor
`pax`	Predictor
`vtype`	Target class

The target has four classes. The class distribution in the reference dataset is Bus: 19 records, Car: 32 records, Truck: 27 records, and Bike: 21 records. The distribution sums to 99 instances.

The dataset is small and structured. Its attributes and labels should be treated as an educational dataset, and its results should not be assumed to represent performance on arbitrary real-world vehicle data.

4.2 Software and implementation

- Programming language: Java 21
- IDE: Eclipse
- Machine-learning library: WEKA weka-stable 3.8.6
- Interface: Java Swing
- Dataset format: ARFF

- Build/dependency management: Maven

The shared model utility loads the dataset, validates that it contains instances and at least two attributes, sets the final attribute as the target, and requires the target attribute to be nominal. The application can evaluate cross-validation, train a model on the full dataset, and perform a holdout evaluation.

4.3 Baseline configuration

Parameter	Value
Classifier	WEKA MultilayerPerceptron
Learning rate (`-L`)	0.3
Momentum (`-M`)	0.2
Epochs (`-N`)	500
Validation set size (`-V`)	0
Classifier seed (`-S`)	0
Validation threshold (`-E`)	20
Hidden layers (`-H`)	`a` (WEKA automatic sizing)
Evaluation	Stratified 10-fold cross-validation
Cross-validation random seed	1

The classifier seed and cross-validation seed serve different purposes: the classifier seed controls the MLP's random initialization/training behavior, while the cross-validation seed controls the randomization used to form folds.

4.4 Learning-rate experiment

The learning rate was changed to 0.1, 0.3, and 0.5. Momentum (0.2), epochs (500), hidden-layer setting (a), classifier seed (0), dataset, number of folds (10), and cross-validation seed (1) were held constant according to the experiment output.

This design makes learning rate the intended independent variable. The measured metrics are accuracy, Kappa, MAE, and RMSE.

5. Results

5.1 Baseline comparison with WEKA Explorer

The Java application's baseline output matched the supplied WEKA Explorer output for the reported metrics and confusion matrix.

Metric	WEKA Explorer reference	Java GUI
Correctly classified	99 / 99	99 / 99
Accuracy	100%	100%
Kappa	1.000	1.000
MAE	0.0184 (rounded)	0.01838
RMSE	0.0271 (rounded)	0.02708

The confusion matrix was:

Actual class	Predicted Bus	Predicted Car	Predicted Truck	Predicted Bike
Bus	19	0	0	0
Car	0	32	0	0
Truck	0	0	27	0
Bike	0	0	0	21

Every record was classified correctly in the cross-validation predictions. The match demonstrates that the Java application can reproduce the supplied reference run under the stated dataset and settings.

The WEKA output also displays a classifier model trained on the full 99-instance dataset. That full-data model is separate from the cross-validation process used to calculate the evaluation metrics. The model weights should not be interpreted as the model from one particular fold.

5.2 Learning-rate comparison

The measured results from the Java application's learning-rate experiment are:

Learning rate	Accuracy	Kappa	MAE	RMSE
0.1	98.99%	0.9863	0.03811	0.06076
0.3	100.00%	1.0000	0.01838	0.02708
0.5	100.00%	1.0000	0.01299	0.01876

5.3 Interpretation

At a learning rate of 0.1, accuracy was 98.99%, with MAE of 0.03811 and RMSE of 0.06076. At learning rates of 0.3 and 0.5, accuracy reached 100% and Kappa reached 1.000. Among the three tested configurations, 0.5 produced the lowest MAE and RMSE.

The results suggest that the learning rate influenced the probability errors in this experiment. Although accuracy was identical at 0.3 and 0.5, the lower MAE and RMSE at 0.5 indicate that the predicted class probabilities were closer to the actual class-indicator values on average under these runs.

The result does not prove that 0.5 is universally optimal. Only three learning rates were tested, and the experiment used one small dataset and one cross-validation seed. A more robust comparison would repeat the experiment with several seeds or use repeated cross-validation, then compare average performance and variability. If model selection is intended to estimate final generalization performance, parameter tuning should also be separated from the final evaluation to avoid optimistic bias.

6. Discussion

The project demonstrates a complete introductory workflow: loading labelled data, configuring an MLP, evaluating predictions, and comparing a training hyperparameter. The Swing GUI makes these operations accessible without requiring the user to edit Java source code for every run.

The perfect baseline accuracy is encouraging but should be interpreted cautiously. With 99 observations, a small dataset can produce optimistic estimates, especially if the classification patterns are simple or the data contain highly distinctive attributes. Further testing on independent data is required before making claims about performance outside this dataset.

The learning-rate experiment also shows why accuracy alone is insufficient. Both 0.3 and 0.5 achieved 100% accuracy, but 0.5 had lower MAE and RMSE. These metrics provide additional information about prediction probabilities, although lower values alone do not establish good calibration or better generalization.

The experiment is controlled in the sense that only the learning rate is intended to change. To strengthen reproducibility, the final submission should retain the dataset, configuration, CSV output, and instructions for running the same experiment. Results should be rerun if the code, WEKA version, dataset, or random seeds change.

7. Limitations and Future Work

7. Dataset size: 99 instances are insufficient to establish robust performance for a broad real-world population.
8. Dataset scope: the data are structured vehicle attributes, not raw images or sensor streams. The project does not perform visual vehicle recognition.
9. Limited parameter range: only three learning rates were compared. More values could be tested using a predefined experiment plan.
10. Randomness: one cross-validation seed was used for the reported comparison. Repeated runs would help quantify variability.
11. External validation: an independent test dataset was not included in the reported reference experiment.
12. Runtime analysis: the current comparison focuses on accuracy and error metrics. A future extension could record training time consistently for each configuration.

8. Conclusion

The project successfully integrates WEKA's Multilayer Perceptron classifier into a Java application with a Swing GUI. The baseline experiment reproduced the supplied WEKA Explorer results on the 99-instance vehicle dataset, achieving 100% accuracy, Kappa 1.000, MAE approximately 0.01838, and RMSE approximately 0.02708 under stratified 10-fold cross-validation.

The learning-rate experiment found that 0.1 achieved 98.99% accuracy, while 0.3 and 0.5 both achieved 100%. The 0.5 configuration yielded the lowest measured MAE and RMSE among the tested values. Within this limited experiment, 0.5 is therefore the strongest candidate according to these metrics, but further validation is needed before generalizing that conclusion.

Overall, the work demonstrates how to implement a classification model, evaluate it systematically, and study the effect of a model hyperparameter. It also highlights the importance of reporting the evaluation method, random seeds, dataset limitations, and measured results.

References

13. University of Waikato. *WEKA: The Waikato Environment for Knowledge Analysis*. https://www.cs.waikato.ac.nz/ml/weka/
14. University of Waikato. *WEKA API Documentation: MultilayerPerceptron*. https://weka.sourceforge.io/doc.stable/weka/classifiers/functions/MultilayerPerceptron.html
15. Rumelhart, D. E., Hinton, G. E., & Williams, R. J. (1986). Learning representations by back-propagating errors. *Nature, 323*, 533–536. https://doi.org/10.1038/323533a0

Appendix A. Reproduction Steps

16. Open the project in Eclipse with Java 21 configured.
17. Allow Maven to resolve weka-stable version 3.8.6.
18. Run WekaProjectGUI.
19. Select vehicledata_99.arff.
20. Set learning rate to 0.3, momentum to 0.2, epochs to 500, hidden layers to a, classifier seed to 0, CV folds to 10, and CV random seed to 1.
21. Click Run Cross-Validation and compare the output with Section 5.1.
22. Click Compare Learning Rates and inspect/export the measured CSV.
23. Retain the exact dataset and output CSV with the submission for reproducibility.

Appendix B. Submission Checklist

- ☐ Replace the course name and submission date on the cover.
- ☐ Add the lecturer's requested formatting or cover page, if applicable.
- ☐ Insert screenshots of the running GUI and experiment output if required.
- ☐ Include the Java source code and the exact 99-record ARFF dataset used for the reported run.
- ☐ Include the measured `learning_rate_experiment.csv`.
- ☐ Confirm the project compiles from a clean Eclipse/Maven workspace.
- ☐ Ensure every reported number corresponds to an actual captured run.